

**DOCUMENTATION
TECHNIQUE
ISANDRE**



Sommaire

1 Introduction

- Objectif du projet
- Description du jeu
- Public cible
- Technologies utilisées

2 Gameplay et Mécaniques

- Contrôles
- Système de tir et projectiles
- IA des ennemis
- Système de spawn
- Gestion des collisions (Layers)
- Progression

3 Architecture du Code

- Structure du projet
- Patrons de conception utilisés
- Diagramme UML simplifié

4 Graphismes et Audio

- Assets graphiques (sprites, animations, effets visuels)
- Sound design et gestion audio

5 Tools, Test unitaires & CI/CD

- Outils de développement
- Test unit
- CI/CD (Intégration Continue & Déploiement Automatisé)

1 Introduction

Objectif

L'objectif du projet est de développer un **Twin Stick Shooter** jouable au **clavier/souris** et à la **manette**, où le joueur peut utiliser différentes armes, aussi bien en **corps à corps** qu'à **distance**. Le projet sera **rigoureusement documenté** afin de permettre à d'autres développeurs de travailler efficacement sur son évolution.

Description du jeu

Le projet est un Twin Stick Shooter jouable au clavier/souris et à la manette. Le joueur affronte des vagues d'ennemis dans un environnement, en alternant entre **attaques à distance** (armes à feu, projectiles) et **corps à corps** (pieds, poings). Le gameplay mise sur la fluidité des déplacements et des combats. L'objectif est de survivre aux vagues d'ennemis.

Public cible

Le jeu vise un public :

- **Joueurs de Twin Stick Shooters** (Nuclear Throne, Enter the Gungeon, Dead Cells)
- **Fans de jeux d'action rapides** nécessitant réflexes et gestion tactique
- **Joueurs PC et console** recherchant une expérience fluide et accessible

Le jeu s'adresse principalement aux **joueurs de 16 ans et plus**, appréciant les défis et la maîtrise des mécaniques de combat.

Technologies utilisées

Moteur de jeu : Unity 6

Langage de programmation : C#

Packages et librairies :

- **DoTween** : Gestion avancée des animations et tweening

- **Cinemachine** : Caméra dynamique et adaptative
- **TextMesh Pro (TMP Pro)** : Affichage de texte amélioré

Gameplay et Mécaniques

Contrôles (Twin-Stick, clavier/souris, manette)

Le jeu utilise un **schéma Twin-Stick**, avec le New System Input de Unity.

Clavier / Souris

- **ZQSD / Flèches** → Déplacement
- **Souris** → Viser
- **Clic gauche** → Attaque
- **Clic droit** → Viser
- **Left Shift** → Courir
- **E** → Interagir
- **Left Ctrl** → Saut rapide

Manette

- **Stick gauche** → Déplacement
- **Stick droit** → Viser & Tirer
- **A / LS / RS** → Saut rapide
- **LT / X** → Courir
- **B** → Interagir

Système de tir et projectiles

Le jeu utilise un **système de tir modulaire** avec des composants pour chaque type de tir, à ajouter à la préfab de votre armes.

Types de Tir

- **Tir simple** : Une balle ou un projectile unique à chaque tir.
- **Tir en rafale** : Plusieurs projectiles sont tirés en succession rapide.
- **Tir à dispersion** : Propagation de plusieurs projectiles (ex : fusil à pompe).

Caractéristiques des Projectiles

- **Vitesse** : Détermine la rapidité du projectile.
- **Taille** : Détermine la taille du projectile.

Chaque projectiles à ses statistiques contenu dans un scriptables objects

IA des ennemis

Comportements de base

- **Marche** : L'ennemi se déplace vers une zone.
- **Attaque** : Une fois le joueur assez proche, il l'attaque.
- **Mort** : Une fois la vie de l'ennemi a 0, il disparaît.

Types d'ennemis

- **Mêlée** : Fonce vers le joueur pour l'attaquer au corps à corps.
- **Distance** : Tire des projectiles et tente de garder ses distances.
- **Tank** : Ennemi lent mais résistant, capable d'encaisser de nombreux dégâts.

Système de Détection

- Fonce vers le joueur pour l'attaquer.

Le système d'IA utilise **NavMesh** pour la navigation et **des états dynamiques** (FSM), chaque ennemi à ses statistiques de base contenu dans un scriptables objects (EntityData).

Système de spawn des ennemis

1. Spawn en dehors du champ de vision

- **Positionnement stratégique** : Les ennemis apparaissent dans des zones **hors du champ de vision** du joueur, ce qui permet de créer un sentiment de surprise et d'urgence.

2. Augmentation des points de vie avec le temps

- **Difficulté croissante** : Les ennemis deviennent progressivement **plus résistants** à mesure que le joueur avance dans le jeu. Cette augmentation est

gérée par un système qui ajuste les **points de vie (PV)** des ennemis après chaque groupe d'ennemis.

- **Calcul dynamique** : L'augmentation des PV est calculée en fonction du nombre d'ennemis déjà tués. Par exemple :
 - Après la 20 ennemis tués, tous les ennemis voient leurs PV augmenter de **20%**.
 - Après la 40 ennemis tués, cette augmentation peut atteindre **40%**, et ainsi de suite.
- **Variabilité selon le type d'ennemi** : Certains ennemis (comme les mini-boss ou les tanks) voient leurs PV augmenter de manière plus marquée.

Gestion des collisions (Layers)

Layers

- **Player** : Le joueur appartient à un layer spécifique pour distinguer ses collisions des autres objets.
- **Enemy** : Les ennemis ont leur propre layer, ce qui permet de définir les collisions spécifiques entre le joueur, les ennemis et d'autres éléments du monde.
- **PlayerBullet / EnemyBullet** : Les projectiles du joueur sont dans un layer dédié pour gérer les interactions de tir.
- **Ground / Wall** : Le sols / murs appartiennent à un layer qui empêche les objets (comme les projectiles) de les traverser.

Progression

Récompenses et équipements : En parallèle de l'augmentation des PV des ennemis, le joueur peut accéder à de nouvelles **armes** après avoir tué un certain nombre d'ennemis lâché par le boss / mini-boss.

3 Architecture du Code

Structure du projet

La structure de projet permet de mieux organiser les ressources afin de les rassembler par catégorie et de les retrouver plus facilement. Les noms de dossiers doivent toujours être en anglais. Certains dossiers provenant de packages externes peuvent se retrouver à la racine du dossier Assets, les basculer dans le dossier External Assets lorsque c'est possible. Le dossier Assets dans Unity doit suivre *au maximum* la structure suivante :

- **Animations** (*contient les Animations/Animator/Avatar Mask/Timelines*)
- **Plugins** (*contient les plugins*)
- **Prefabs** (*contient les prefabs utilisés dans le jeu*)
- **ExternalAssets** (*contient tous les éléments d'assets artistiques*)
 - o **Audio** (*contient les fichiers audio pour la musique/bruitages/Audio Mixer*)
 - o **Fonts** (*contient les Polices d'écritures/Atlas TextMeshPro*)
 - o **Materials** (*contient les matériaux de modé
3D/Shader/Skybox/Particules/UI/Sprites/etc*)
 - o **Models** (*contient les modèles 3D*)
 - o **Shaders** (*contient les shaders*)
 - o **Sprites** (*contient les sprites et spritesheets 2D/UI/Logo/Icônes*)
 - o **Textures** (*contient les textures pour les
matériaux/RenderCamera/RawTexture*)
 - o **Video** (*contient les vidéos*)
 - o **PhysicMaterials** (*contient les matériaux physiques 2D ou 3D*)
- **Scenes** (*contient les scènes du jeu*)
- **Settings** (*contient les profils Post Process, Presets, etc*)
- **Scripts** (*contient les scripts du projet*)
- **ScriptableObjects** (*contient les scriptables du projet*)

Les règles de nommage des fichiers permettent de retrouver facilement un élément lors de la recherche rapide dans l'éditeur ou l'assignation dans un composant. Les noms de fichiers doivent toujours être en anglais et suivent *au maximum* le modèle suivant :

Type_Category_Element_Variation

Voici une liste non exhaustive des différents types que l'on peut retrouver dans un projet :

NOM	TYPE	EXEMPLES
Audio	Aud	<i>Aud_Music_GameTheme_Main</i> <i>Aud_SFX_PlayerAttack_Bow</i>
Fonts	Fon	<i>Fon_Arial_Bold</i>
Material	Mat	<i>Mat_Skybox_Moon</i> <i>Mat_Player_Body_Iron</i> <i>Mat_FX_Projectile_Fireball</i>
Prefab	Pre	<i>Pre_FX_Monster_Death</i> <i>Pre_UI_GameCanvas</i>
Model	Mod	<i>Mod_Animal_Bird_Little</i> <i>Mod_Character_Player</i>
Shader	Sha	<i>Sha_Toon_Grayscale</i> <i>Sha_Flat_Water_</i>
Sprite	Spr	<i>Spr_UI_Button_Quit</i> <i>Spr_Item_Coin</i>
SpriteSheet	SprSht	<i>SprSht_Character_Player_Run</i>
Texture	Tex	<i>Tex_Render_Layer_Fade</i>
Video	Vid	<i>Vid_Intro</i>

La nomenclature du code suit des conventions strictes pour garantir la lisibilité, la maintenabilité et la cohérence du projet. Voici les règles à suivre pour le nommage des éléments dans le code :

1. Classes

- **Nom** : Le nom d'une classe doit être en **PascalCase** (première lettre en majuscule, chaque mot suivant commence par une majuscule).
 - Exemple : `PlayerCharacter`, `EnemyAI`, `WeaponManager`

2. Variables

- **Nom** : Les variables doivent utiliser le **camelCase** (première lettre en minuscule, chaque mot suivant commence par une majuscule).
 - Exemple : `playerHealth`, `enemyCount`, `weaponDamage`

3. Constantes

- **Nom** : Les constantes doivent être en **Screaming Snake Case** avec des underscores pour séparer les mots.
 - Exemple : `MAX_HEALTH`, `INITIAL_DAMAGE`

4. Énumérations (Enums)

- **Nom** : Le nom des énumérations doit être en **PascalCase**, et chaque valeur de l'énumération en **Pascal Case**.
 - Exemple :

```
enum WeaponType {  
    MeleeSword,  
    RangedBow,  
    MagicWand  
}
```

5. Structures

- **Nom** : Le nom des structures doit être en **PascalCase**, avec des noms de variables en **camelCase**.
 - Exemple :

```
struct WeaponStats {  
    int damage;  
    float range;  
}
```

7. Fonctions

- **Nom** : Le nom des fonctions doit être en **PascalCase**, avec un verbe à l'infinitif pour décrire l'action.

Exemple : `MovePlayer()`, `FireProjectile()`, `ReloadWeapon()`

Patrons de conception utilisés

Singleton

- **Utilisation** : Le **patron Singleton** est utilisé pour les **gestionnaires de systèmes globaux** tels que :
 - GameManager :
 - UI Manager :
 - Time Manager
 - Camera Manager
 - Audio Manager
 - Pool Manager
 - ResourceObjectHolder

State Machine (Machine à États)

- **Utilisation** : Le **patron State Machine** est utilisé pour gérer les différents états de l'**IA des ennemis**.

Component

- Le **patron Component** permet de **modulariser** le comportement des armes en séparant leurs différentes façons de tir sous forme de **composants indépendants**.

Diagramme UML

Lien du UML :

https://drive.google.com/drive/folders/1XUnrzfzyHOaGx_ywhRLFbMAu8MVq48FG?usp=sharing

A ouvrir sur draw.io

4 Graphismes et Audio

Assets graphiques

1. Assets Graphiques (Sprites, Animations, Effets Visuels)

3D - Personnages et Environnement :

- **Mini Arcade** : Modèles 3D pour personnages et environnement
[Lien vers Mini Arcade \(Kenney\)](#)

3D - Armes :

- **Blaster Kit** : Ensemble de modèles d'armes, comprenant des armes à feu et autres projectiles.
[Lien vers Blaster Kit \(Kenney\)](#)

Cartoon FX Free :

- **Cartoon FX** est une bibliothèque d'effets visuels de style cartoon, idéale pour des explosions, des tirs et d'autres effets stylisés.
[Lien vers Cartoon FX Free \(Jean Moreno\)](#)

Sci-fi Arsenal :

- Effets visuels futuristes incluant des particules, des explosions et des effets lumineux.
[Lien vers Sci-fi Arsenal \(Unity Asset Store\)](#)

Sound design et gestion audio

Musique et Effets Sonores :

- **YouTube Audio Library** : Pour la musique et les effets sonores.
[Lien vers YouTube Audio Library](#)

5 Tools, Test unitaires & CI/CD

Outils de développement

- **Moteur de jeu** : Unity 6
- **IDE** : Visual Studio / Rider
- **Versioning** : Git + GitHub
- **Gestion de projet** : Trello
- **Profilage & Optimisation** : Unity Profiler, Deep Profiling
- **Debug UI** : Fenêtre custom pour l'édition des **ScriptableObjects** en runtime

Fenêtre UI Custom pour ScriptableObjects (SO)

Une fenêtre d'édition **UI custom** a été développée pour permettre la modification des valeurs des **ScriptableObjects (SO)** directement dans l'éditeur Unity.

♦ Fonctionnalités :

- ✓ Modification des **valeurs en temps réel** sans passer par l'inspecteur Unity
- ✓ **Sauvegarde et chargement de presets** pour tester différentes configurations
- ✓ **Renommage et suppression** de presets directement via l'UI

Tests Unitaires

📌 Types de tests implémentés :

- **Player**
 - Déplacements
 - Dash
 - Vie
- **Armes**
 - Coup de pied
 - Coup de poing
 - Pistolet
 - Fusil a rafale
 - Fusil a pompe

CI/CD (Intégration Continue & Tests Automatisés)

Pour automatiser l'exécution des tests et le déploiement, un pipeline **GitHub Actions** est mis en place.

1 Déclencheurs (Triggers)

- Lors d'un push ou pull request sur `dev`

2 Étapes du Pipeline

1. **Checkout du repo**
2. **Installation de Unity**
3. **Exécution des tests unitaires**
4. **Compilation du projet**
5. **Génération et upload du build**